Introduction to the Theory of Computation

Lecture 7. Regular Expressions

Neo (Yuan) Li

Oct 2026

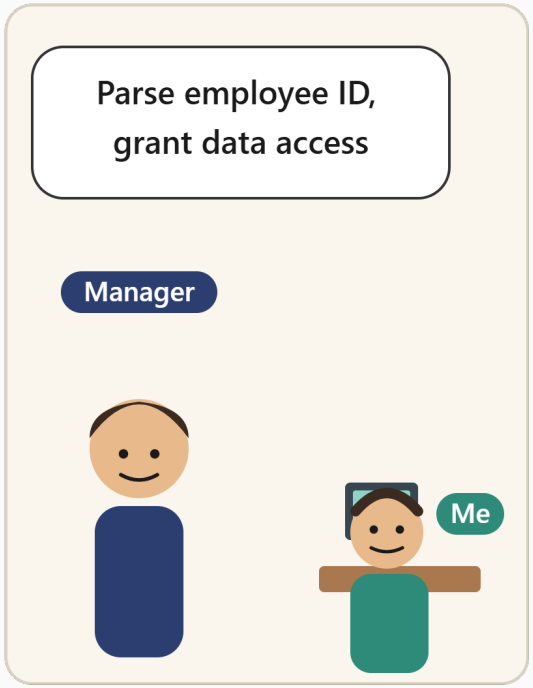
Fudan University

yuan_li@fudan.edu.cn

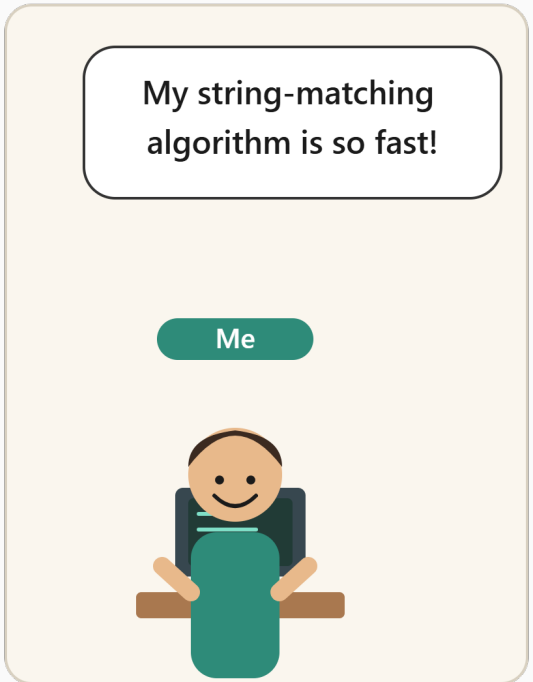
Why Regular Expressions?

- Regular expressions provide an intuitive way to describe patterns in strings.
- Practical regex tools, built on (and extending) regular expressions, are widely used for tasks such as text search, data extraction, and input validation.
- Regular expressions describe exactly **regular languages**.

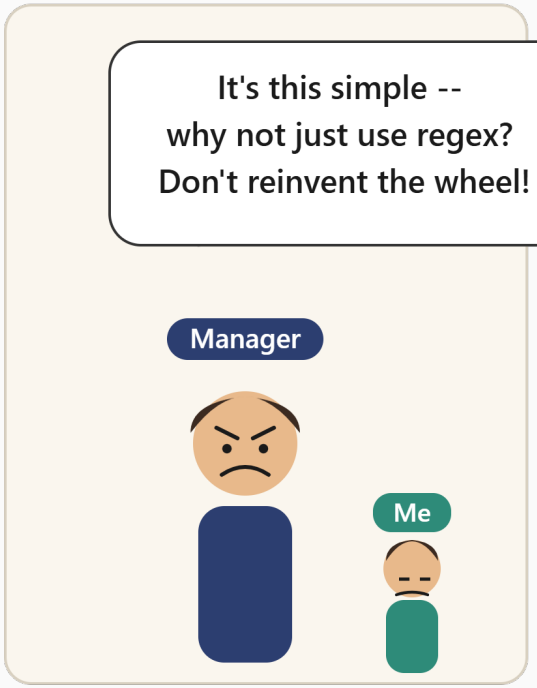
Don’t Reinvent the Wheel



the assignment



reinventing the wheel



why not regex?

Regular Expressions in Practice (1/3)

Open **PowerShell** and try this.

Binary strings starting with 0:

```
PS> '0101' -match '^0[01]*$'
True
PS> '1010' -match '^0[01]*$'
False
```

- `[01]` means $0 \cup 1$, and `*` is the star.
- `^` and `$` anchor the start and end, so the *whole* string must match.

Regular Expressions in Practice (2/3)

Validating a Fudan email address:

```
PS> 'yuan_li@fudan.edu.cn' -match '^[\w.]+@fudan\.edu\.
    cn$'
True
```

- `[\w.]+` means one or more letters, digits, underscores, or dots.
- `\.` is a literal dot.

Regular Expressions in Practice (3/3)

Finding an IPv4 address in text:

```
PS> "request from 192.168.1.15" -match '\d{1,3}\.\d
    {1,3}\.\d{1,3}\.\d{1,3}'
True
```

- `\d{1,3}` matches 1 to 3 digits; four such blocks are glued together with dots.
- No `^` or `$` here: we only ask whether the string **contains** an address somewhere.

Definition: Regular Expression

Definition 1

R is a *regular expression* if R is

1. a symbol s, for some $s \in \Sigma$,
2. ϵ,
3. $\varnothing$,
4. $(R_1 \cup R_2)$, where R_1, R_2 are regular expressions,
5. $(R_1 R_2)$, where R_1, R_2 are regular expressions,
6. $(R_1)^*$, where R_1 is a regular expression.

Definition: Regular Expression

Definition 2

The *language* $L(R)$ of a regular expression R is defined by

1. $L(s) = \{s\}$, for $s \in \Sigma$,
2. $L(\epsilon) = \{\epsilon\}$,
3. $L(\varnothing) = \varnothing$,
4. $L(R_1 \cup R_2) = L(R_1) \cup L(R_2)$,
5. $L(R_1 R_2) = L(R_1) \circ L(R_2)$,
6. $L(R_1^*) = L(R_1)^*$.

Examples of Regular Expressions

- $0^*10^* = \{w : w \text{ contains a single } 1\}$
- $\Sigma^*1\Sigma^* = \{w : w \text{ contains at least one } 1\}$
- $(\Sigma\Sigma)^* = \{w : w \text{ has even length}\}$
- $0\Sigma^*0 \cup 1\Sigma^*1 \cup 0 \cup 1$
  $= \{w :$
  w starts and ends with the same symbol$\}$

In-Class Exercise: Write a Regular Expression

Write a regular expression over $\Sigma = \{0, 1\}$ for each language:

1. w contains the substring 110
2. w does not contain 00 as a substring
3. the number of 1's in w is a multiple of 3
4. w contains at least two 1's and one 0
5. w starts and ends with different symbols
6. the second-to-last symbol of w is 1

Theorem: Regular Expressions = Regular Languages

Theorem 3

A language is regular if and only if some regular expression describes it.

We prove both directions:

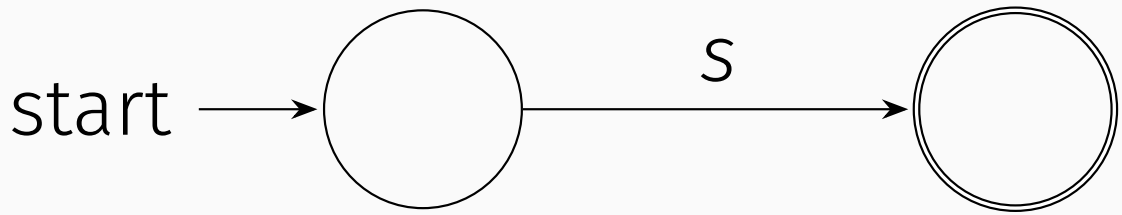
- regular expression $\Rightarrow$ NFA
- DFA/NFA $\Rightarrow$ regular expression

Direction 1: Regular Expression $\Rightarrow$ NFA

Idea: build the NFA by induction, following how the regular expression is built.

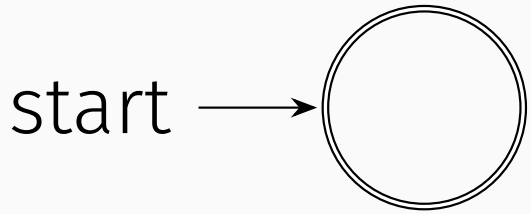
Three base cases (a single symbol, ϵ, $\varnothing$), and three ways to combine smaller NFAs into bigger ones (union, concatenation, star) – exactly the closure constructions we already proved.

Base Case 1: A Single Symbol

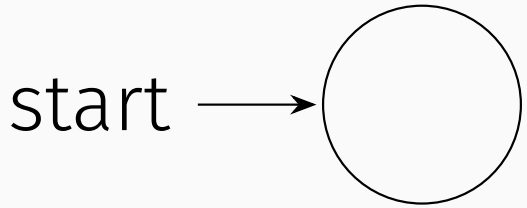


$$R = s$$

Base Cases 2 & 3: The Empty String and the Empty Language



$R = \epsilon$



$R = \varnothing$

Inductive Cases

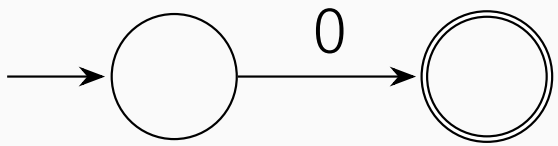
For $R = R_1 \cup R_2$, $R = R_1 R_2$, and $R = R_1^*$, we build the NFA using exactly the union, concatenation, and star constructions we already proved earlier.

Since every regular expression is built from the three base cases using these three operations, this gives an NFA for **any** regular expression.

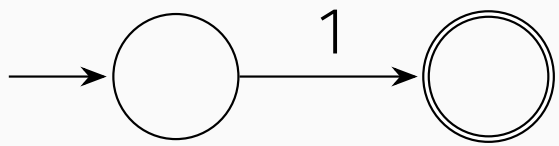
Example: Regular Expression $\Rightarrow$ NFA (1/4)

Convert $(01 \cup 1)^*$ into an NFA, building from the inside out.

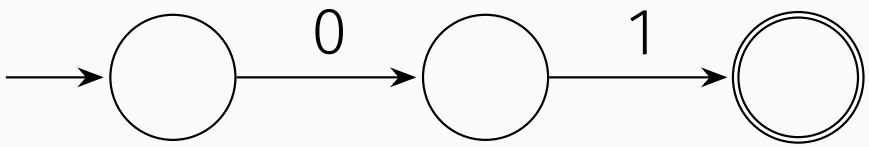
Base cases 0 and 1, then concatenation 01:



$R = 0$



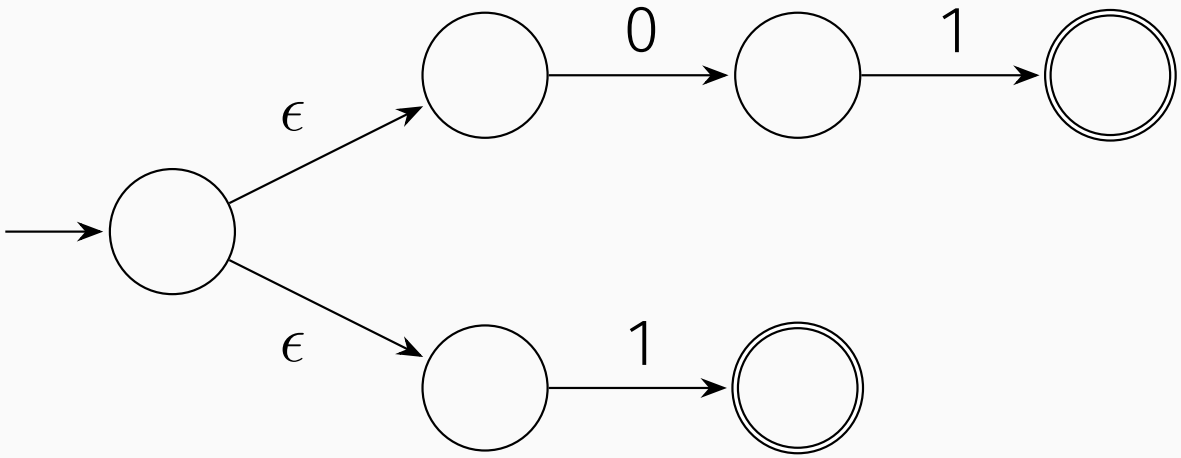
$R = 1$



$R = 01$

Example: Regular Expression $\Rightarrow$ NFA (2/4)

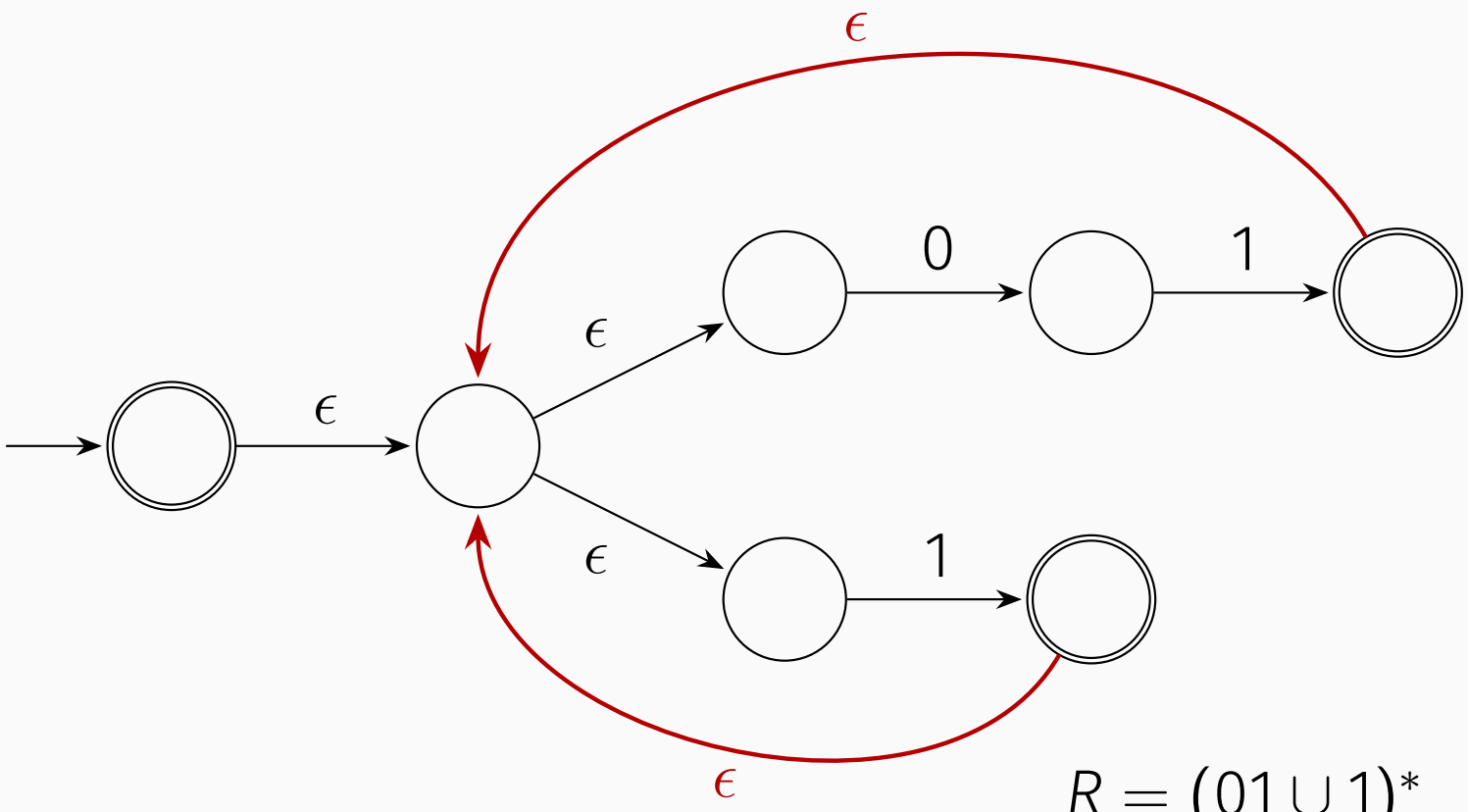
Union $01 \cup 1$: a new start state with ϵ-arrows to both machines.



$$R = 01 \cup 1$$

Example: Regular Expression ⇒ NFA (3/4)

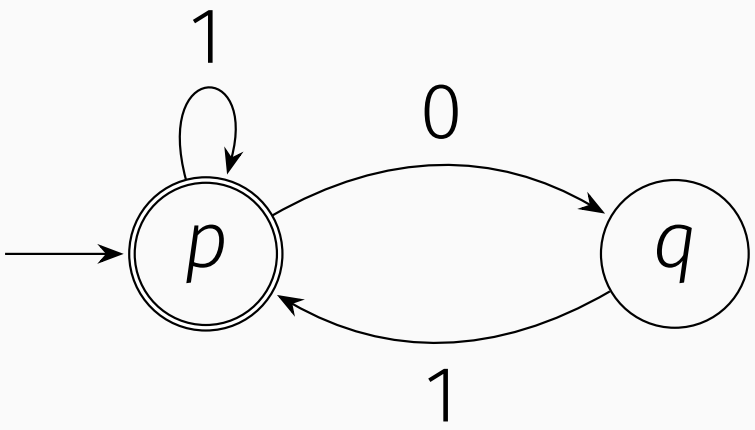
Star $(01 \cup 1)^*$: a new start state that is also accepting, plus ϵ-arrows from accept states back to the old start.



$R = (01 \cup 1)^*$

Example: Regular Expression $\Rightarrow$ NFA (4/4)

A minimal NFA for $(01 \cup 1)^*$ has only 2 states:



Direction 2: DFA/NFA $\Rightarrow$ Regular Expression

Proof idea:

- Convert the NFA into a **GNFA**: arrows are labeled by regular expressions, not just symbols.
- Remove states one by one, updating labels, until only start and accept remain.
- The label on the last arrow is the answer.

GNFA: Formal Definition

Definition 4
A *generalized NFA* (GNFA) is a 5-tuple $(Q, \Sigma, \delta, q_{\text{start}}, q_{\text{accept}})$, where

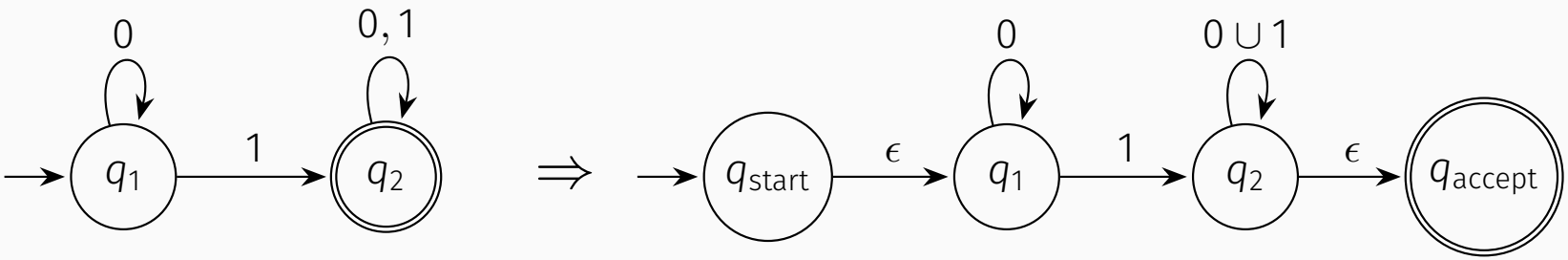
1. Q is a finite set of states,
2. Σ is the input alphabet,
3. $\delta : \big(Q \setminus \{q_{\text{accept}}\}\big) \times \big(Q \setminus \{q_{\text{start}}\}\big) \to \mathcal{R}$ is the transition function, where $\mathcal{R}$ is the set of regular expressions over Σ,
4. $q_{\text{start}} \in Q$ is the start state,
5. $q_{\text{accept}} \in Q$ is the accept state, $q_{\text{accept}} \neq q_{\text{start}}$.

Standard form (encoded in the domain of δ):

- no arrows into q_{start}, no arrows out of q_{accept};
- exactly one arrow between every other pair of states (self-loops included); a missing arrow is labeled $\varnothing$.

Step 1: Converting an NFA into a GNFA

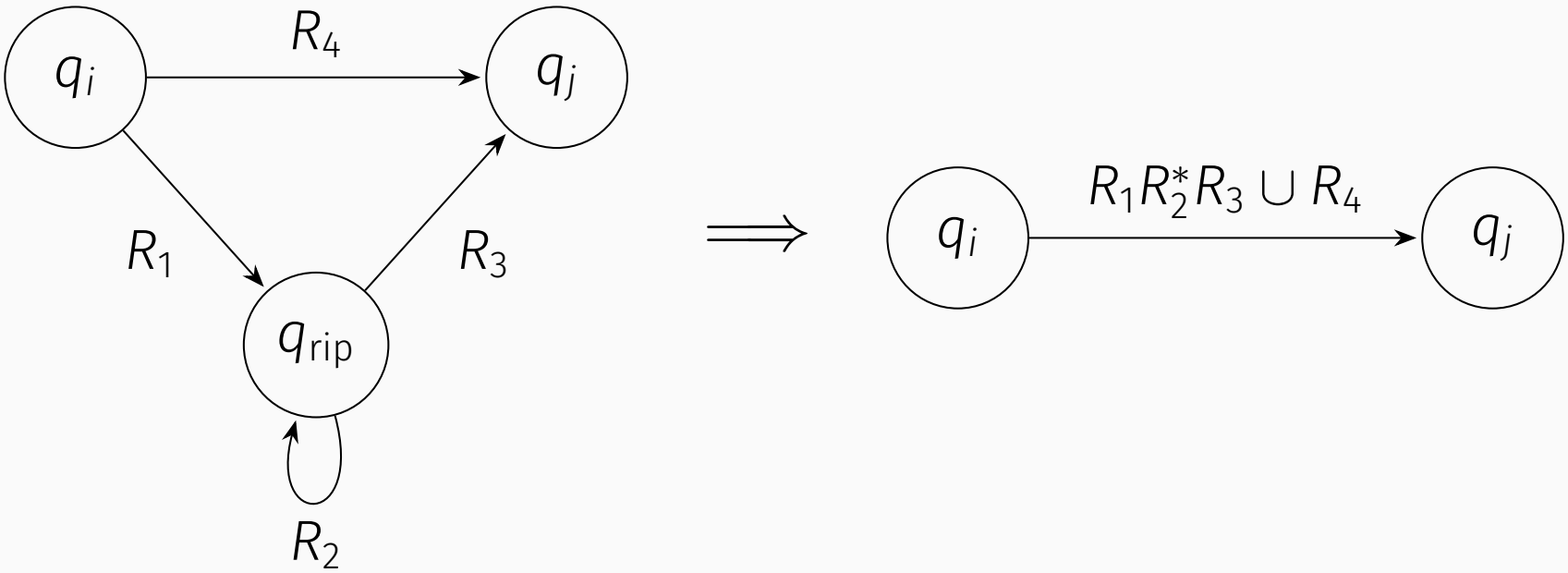
1. Add a new start state, with an ϵ-arrow to the old start state.
2. Add a new accept state, with ϵ-arrows from every old accept state (which are no longer accepting).
3. Merge multiple labels on the same arrow into a union: $0, 1 \Rightarrow 0 \cup 1$.
4. Add every missing arrow, labeled $\varnothing$ (usually not drawn).



Step 2: Remove States One by One

Pick **any** state q_{rip} other than q_{start} and q_{accept}, and remove it.

For every pair q_i, q_j, update the label to cover all paths through q_{rip}:



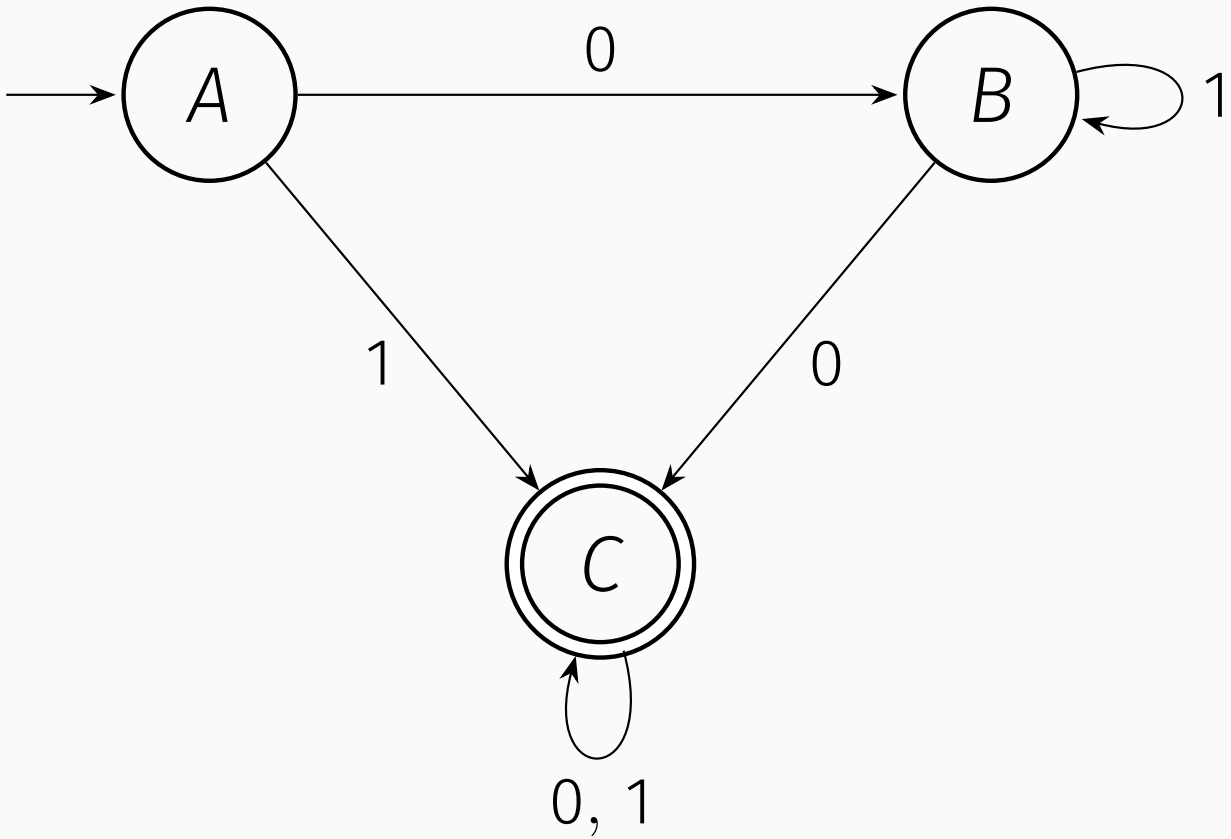
Step 3: Finishing Up

Repeat until only q_{start} and q_{accept} remain.

The label on the last arrow is a regular expression for $L(M)$.

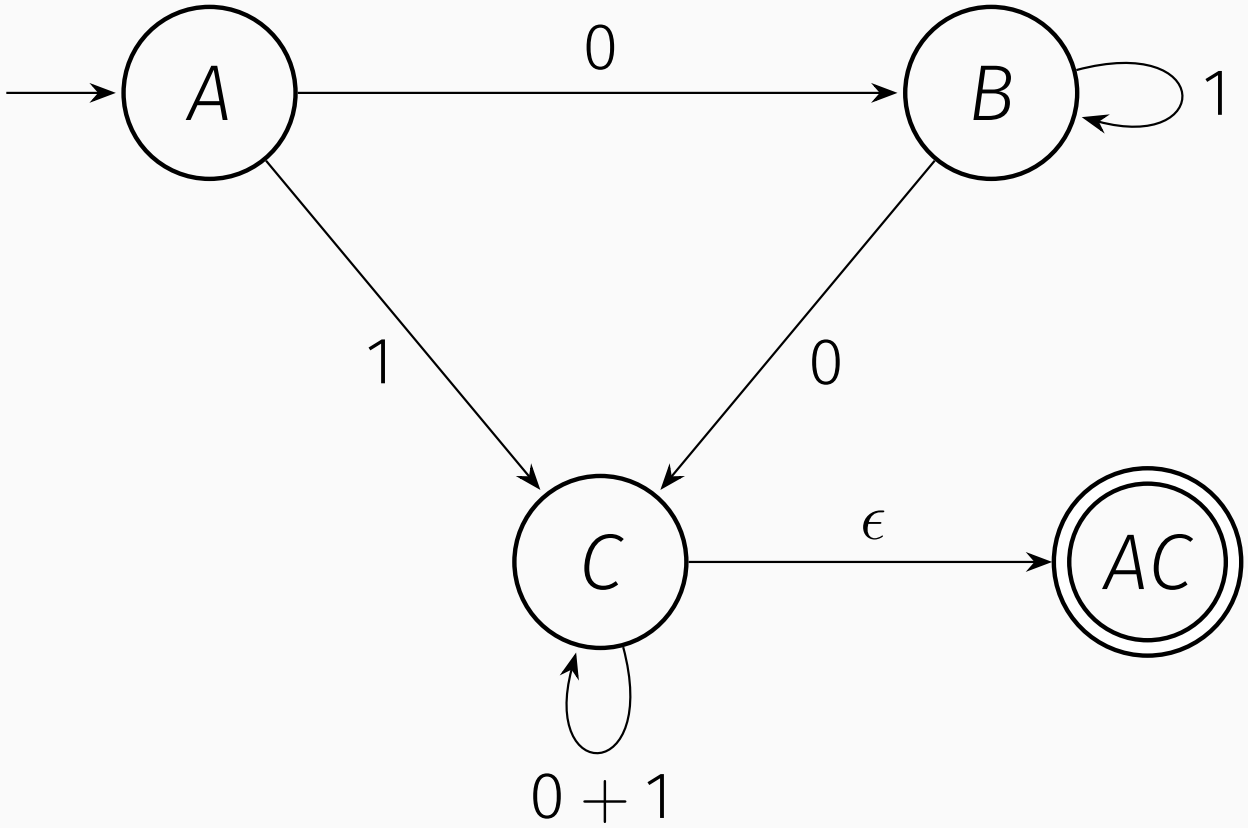
Example: NFA $\Rightarrow$ Regular Expression (1/6)

We start with a 3-state NFA over $\Sigma = \{0, 1\}$, with start state A and accept state C.



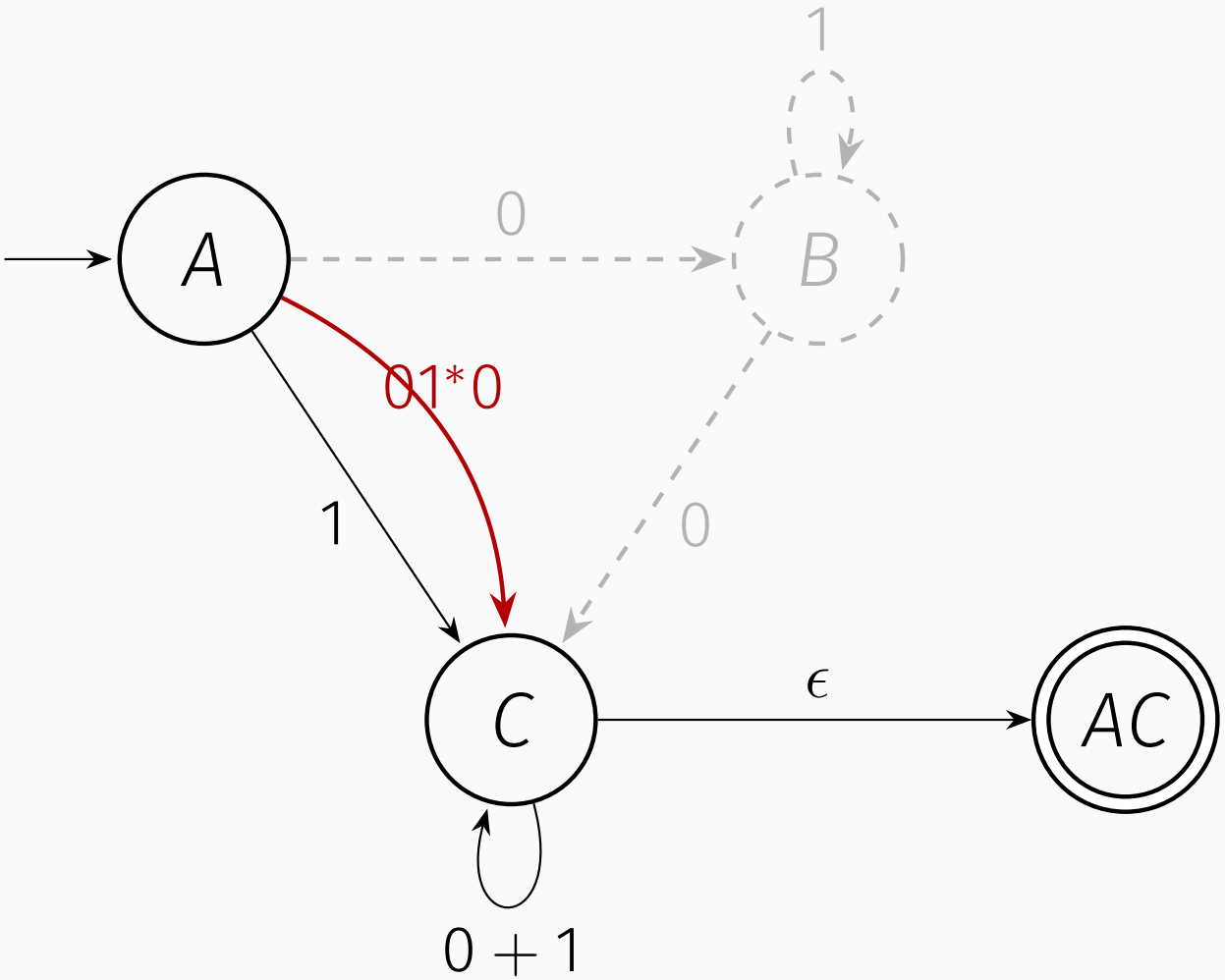
Example: NFA $\Rightarrow$ Regular Expression (2/6)

A already has no incoming arrows, so it can serve as the GNFA start state directly. We still need a unique accept state with no outgoing arrows, so add *AC*.



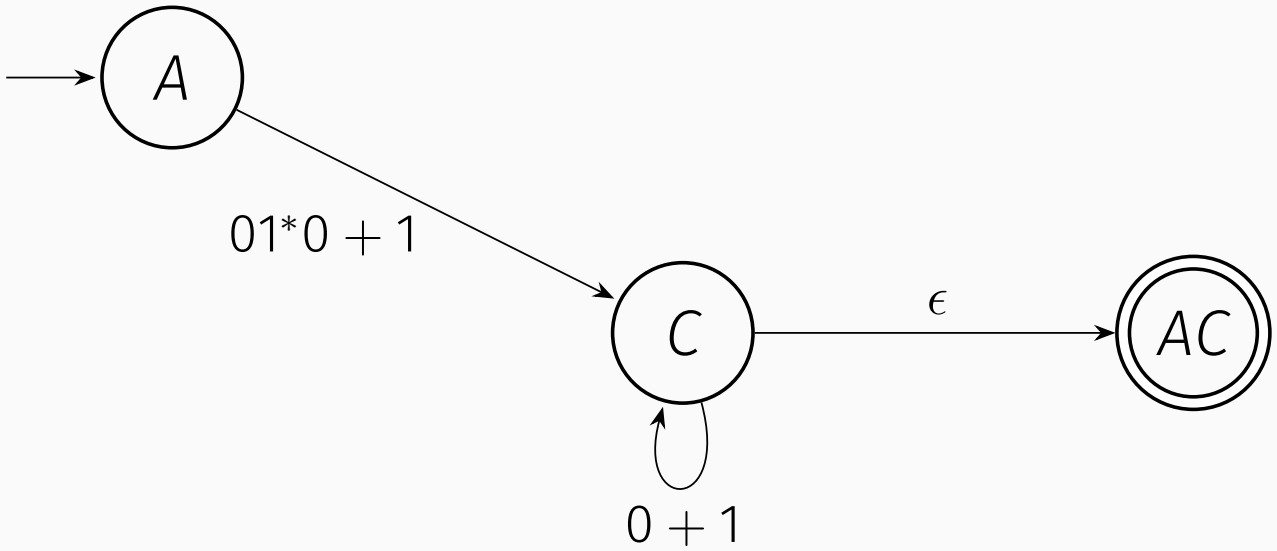
Example: NFA $\Rightarrow$ Regular Expression (3/6)

B has a self-loop, so the replacement label is 01*0.



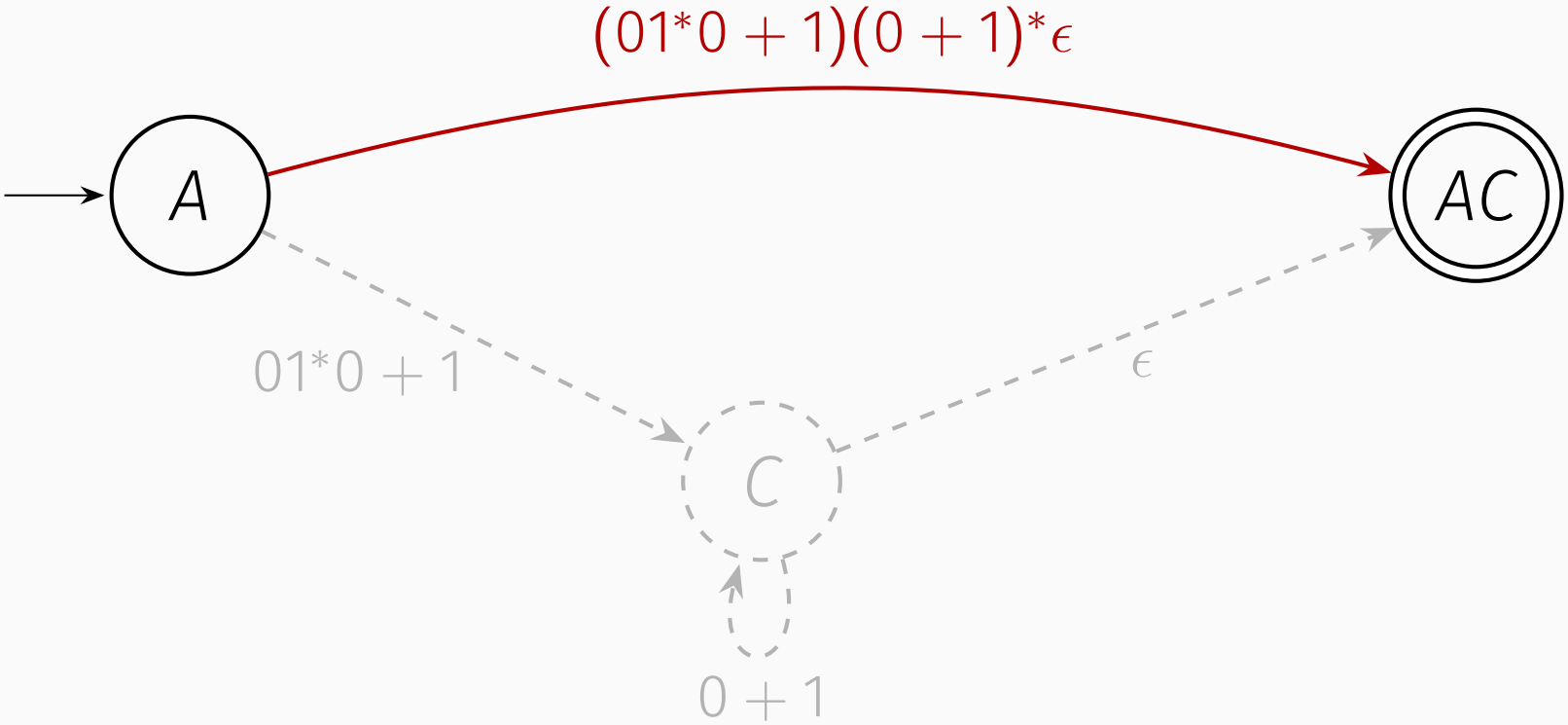
Example: NFA $\Rightarrow$ Regular Expression (4/6)

Down to three states: A, C, AC. The two parallel labels into C are combined with a union.



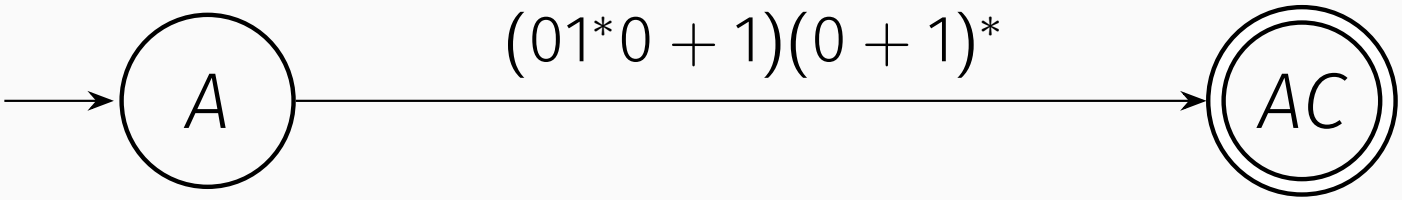
Example: NFA ⇒ Regular Expression (5/6)

C also has a self-loop, so another Kleene star appears; this is the last state to remove.



Example: NFA $\Rightarrow$ Regular Expression (6/6)

Only A and AC remain – the label on the single arrow **is** the regular expression for the language.



$$L = (01^*0 + 1)(0 + 1)^*$$

Why Does Removing a State Preserve the Language?

Let G be a GNFA. Pick any $q_{\text{rip}} \notin \{q_{\text{start}}, q_{\text{accept}}\}$.
Let G' be G with q_{rip} removed: $Q' = Q \setminus \{q_{\text{rip}}\}$,
and for all $q_i \in Q' \setminus \{q_{\text{accept}}\}$, $q_j \in Q' \setminus \{q_{\text{start}}\}$,

$$\delta'(q_i, q_j) = \underbrace{\delta(q_i, q_{\text{rip}})}_{R_1}\underbrace{\big(\delta(q_{\text{rip}}, q_{\text{rip}})\big)}_{R_2}^{*}\underbrace{\delta(q_{\text{rip}}, q_j)}_{R_3} \cup \underbrace{\delta(q_i, q_j)}_{R_4}.$$

Here $q_i = q_j$ is allowed.

Goal: $L(G) = L(G')$.

Key Idea: Collapse and Expand

Compare accepting runs of G and G':

- In G, a visit to q_{rip} looks like: enter (R_1), loop any number of times (R_2), leave (R_3).
- In G', this whole detour is a **single step**, labeled $R_1 R_2^* R_3$.

So we can translate runs in both directions:

- $L(G) \subseteq L(G')$: **collapse** each detour into one step.
- $L(G') \subseteq L(G)$: **expand** each such step back into a detour.

Claim 1: $L(G) \subseteq L(G')$

Let $w \in L(G)$. Then $w = w_1 w_2 \cdots w_k$, and G has a run

$$q_{\text{start}} = q_0 \xrightarrow{w_1} q_1 \xrightarrow{w_2} \cdots \xrightarrow{w_k} q_k = q_{\text{accept}}, \qquad w_i \in L\big(\delta(q_{i-1}, q_i)\big).$$

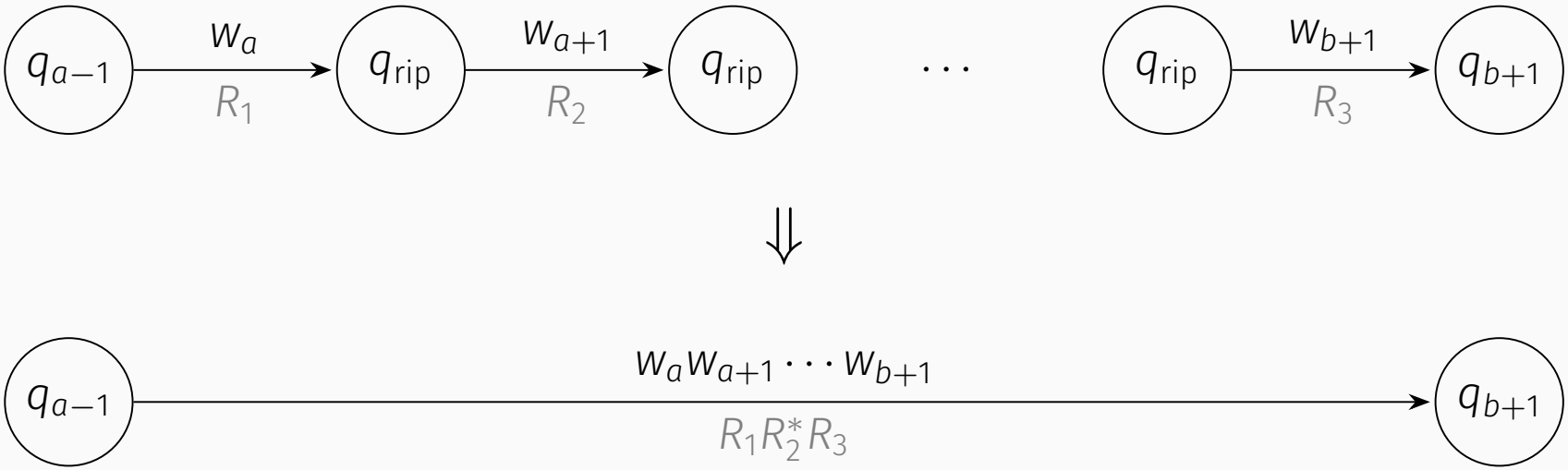
Case 1: the run never visits q_{rip}.

- All q_i are states of G'.
- $L(\delta(q_{i-1}, q_i)) \subseteq L(\delta'(q_{i-1}, q_i))$, since $\delta(q_{i-1}, q_i)$ is the R_4 part of δ'.
- So the same run is an accepting run of G'.

Claim 1, Case 2: Collapse Each Detour

Case 2: the run visits q_{rip}.

Group the consecutive q_{rip}'s in the run into blocks, each extended as far as possible. For one such block $q_a = \cdots = q_b = q_{\text{rip}}$, we have $q_{a-1} \neq q_{\text{rip}}$ and $q_{b+1} \neq q_{\text{rip}}$; both exist since q_{rip} is neither the start nor the accept state.



Claim 1, Case 2: Collapse Each Detour

Here $R_1 = \delta(q_{a-1}, q_{\text{rip}})$, $R_2 = \delta(q_{\text{rip}}, q_{\text{rip}})$, $R_3 = \delta(q_{\text{rip}}, q_{b+1})$, and there are $b - a \geq 0$ loops. So

$$w_a \cdots w_{b+1} \in L(R_1 R_2^* R_3) \subseteq L\big(\delta'(q_{a-1}, q_{b+1})\big).$$

Collapse every such block; the result is an accepting run of G'.

Claim 2: $L(G') \subseteq L(G)$

Let $w \in L(G')$, with an accepting run of G':

$$q_{\text{start}} = q_0 \xrightarrow{w_1} q_1 \xrightarrow{w_2} \cdots \xrightarrow{w_k} q_k = q_{\text{accept}},$$

where each step satisfies

$$w_i \in L\big(\delta'(q_{i-1}, q_i)\big) = L(R_1 R_2^* R_3) \cup L(R_4).$$

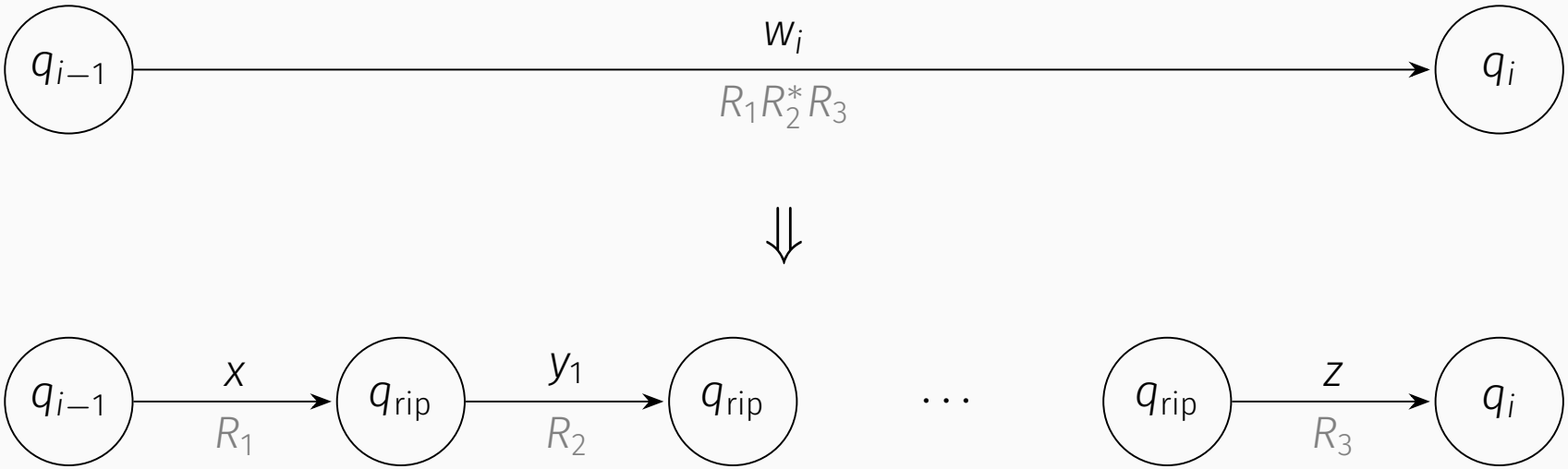
For each step, two cases:

- $w_i \in L(R_4)$: keep it; it is already a step of G.
- $w_i \in L(R_1 R_2^* R_3)$: it is a shortcut through q_{rip}; **expand** it.

Claim 2: Expand Each Shortcut

If $w_i \in L(R_1 R_2^* R_3)$, split it as

$$w_i = x\, y_1 y_2 \cdots y_u\, z, \qquad x \in L(R_1),\ y_1, \ldots, y_u \in L(R_2),\ z \in L(R_3),\ u \geq 0.$$



That is, insert $u + 1$ copies of q_{rip} between q_{i-1} and q_i. Expanding every shortcut gives an accepting run of G.

Putting It Together

- The two claims give $L(G) = L(G')$.
- G' is again a GNFA, with one fewer state.
- By induction on the number of states, removing states until only q_{start} and q_{accept} remain preserves the language.

A Brief History of Finite Automata (1/2)

- **1936** – Turing (and independently Church) formalizes the notion of computation itself, via the Turing machine.
- **1943** – McCulloch & Pitts give the first description of finite automata, as a simplified model of neurons in the brain.
- **1951** – Kleene proves that the languages described by regular expressions are exactly the languages recognized by finite automata.
- **1955–56** – Mealy and Moore independently generalize finite automata to machines with output, now named after them.

A Brief History of Finite Automata (2/2)

- **1956** – Chomsky places regular languages at the bottom of his hierarchy of formal languages, linking automata theory to linguistics.
- **1959** – Rabin & Scott introduce the NFA, and prove it's equivalent in power to the DFA.
- **1968** – Ken Thompson gives an efficient regex-to-NFA construction; it later powers the Unix tool *`grep`*.
- **Today** – finite automata underlie text search, lexical analysis in compilers, and protocol/hardware verification.

References

- M. Sipser. *Introduction to the Theory of Computation*, 3rd ed. Cengage Learning, 2013. Section 1.3.
- W. S. McCulloch and W. Pitts. A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 5:115–133, 1943.
- S. C. Kleene. Representation of events in nerve nets and finite automata. In *Automata Studies*, Princeton University Press, 1956.
- M. O. Rabin and D. Scott. Finite automata and their decision problems. *IBM Journal of Research and Development*, 3(2):114–125, 1959.
- J. A. Brzozowski and E. J. McCluskey. Signal flow graph techniques for sequential circuit state diagrams. *IEEE Transactions on Electronic Computers*, EC-12(2):67–76, 1963.
- K. Thompson. Regular expression search algorithm. *Communications of the ACM*, 11(6):419–422, 1968.